

Cancer Stage Prediction

A Machine Learning Approach to Classifying Cancer Disease Stage

Final Project Report — CBIO313

Global Cancer Patients 2015–2024 Dataset

Prepared by

Merolla Khalifa

Student ID: 231001407

Under the supervision of

Dr. Mohamed ElSayeh

June 2026

Table of Contents

Table of Contents	2
1. Introduction	4
1.1 Background and Motivation	4
1.2 Research Question	4
2. Project Objectives	4
3. Dataset Description.....	5
4. Data Cleaning and Preprocessing	5
Step 1 — Drop Identifier Columns	5
Step 2 — Remove Duplicate Rows.....	5
Step 3 — Handle Missing Values.....	6
Step 4 — Outlier Removal (IQR Method).....	6
Step 5 — Encode the Target Variable	6
Step 6 — Encode Categorical Features	6
Post-Cleaning Audit.....	6
5. Exploratory Data Analysis	6
5.1 Target Variable Distribution (Count Plot & Pie Chart)	6
5.2 Univariate Analysis: Age Distribution (Histogram + KDE)	7
5.3 Bivariate Analysis: Box Plots and Violin Plot	7
5.4 Bivariate Analysis: Cancer Type vs Stage and Treatment Cost by Type.....	7
5.5 Multivariate Analysis: Correlation Heatmap and Pair Plot	7
5.6 EDA Summary	7
6. Feature Engineering and Selection	8
6.1 New Features	8
6.2 Feature Selection	8
7. Model Building and Tuning	8
7.1 Train/Test Split.....	8
7.2 Algorithms Used.....	9
7.3 Initial Model Comparison.....	9
7.4 Hyperparameter Tuning.....	9
8. Validation and Evaluation	10
8.1 What is Validation, and Why Does it Matter?	10
8.2 Cross-Validation Results.....	10
8.3 Final Test-Set Results (Tuned Random Forest)	11

8.4 Confusion Matrix.....	11
9. Discussion: Interpreting the Results	11
9.1 Evidence That the Target Carries No Learnable Signal	11
9.2 Likely Explanation	12
9.3 Why This Is Still a Valid and Useful Finding.....	12
10. Limitations and Recommendations	12
10.1 Limitations.....	12
10.2 Recommendations for Future Work	12
11. Deployment.....	13
12. Conclusion.....	13
References	14

1. Introduction

1.1 Background and Motivation

Cancer remains one of the leading causes of death worldwide, and the stage at which a tumour is diagnosed is one of the strongest determinants of treatment options and survival outlook. Stage 0 and Stage I cancers are typically localised and highly treatable, whereas Stage III and Stage IV cancers are more advanced, often metastatic, and associated with substantially lower survival rates. Because manual staging relies on a combination of imaging, laboratory results, and clinical judgement, the process can be slow, resource-intensive, and inconsistent across institutions.

Machine learning offers a complementary approach: by learning patterns from historical patient records, a trained model can provide a rapid, data-driven estimate of disease stage that can support — though not replace — clinical decision-making. This project explores whether such a model can be built using a combination of demographic, environmental, lifestyle, and clinical attributes.

1.2 Research Question

The central research question guiding this project is:

“Can we accurately classify a cancer patient’s disease stage (Stage 0, I, II, III, IV) using combined demographic, environmental, lifestyle, and clinical parameters?”

This was broken down into four supporting sub-questions, which structured the exploratory and modelling phases of the analysis:

- Which features (e.g. age, cancer type, treatment cost, survival years) are most predictive of cancer stage?
- Do lifestyle and environmental factors (smoking, alcohol use, air pollution, obesity) outweigh clinical measurements in predictive importance?
- Can precision and recall of at least 0.30 be achieved across all five stage classes?
- How do early-stage (0/I) patient profiles differ from late-stage (III/IV) profiles?

2. Project Objectives

Three concrete objectives were set at the start of the project, in line with the course requirements:

1. Build a machine learning model capable of predicting cancer outcomes (specifically, cancer stage) from patient-level data.
2. Analyse the patient dataset thoroughly to identify which features carry the most useful information for prediction.
3. Compare multiple machine learning algorithms — including at least one tuning step — and select the best-performing model for deployment.

These objectives map directly onto the six-step project pipeline that was followed: Project Understanding → Data Mining & Processing → Data Exploration (EDA) → Feature Engineering & Selection → Model Building & Tuning → Interpretation & Deployment.

3. Dataset Description

The dataset used is the “Global Cancer Patients 2015–2024” dataset, publicly available on Kaggle.

Source: kaggle.com/datasets/zahidmughal2343/global-cancer-patients-2015-2024

The raw dataset contains 50,000 patient records across 15 columns, covering the years 2015–2024. A summary of the columns is provided below, grouped by category.

Category	Column(s)	Description
Identifier	Patient_ID	Unique patient identifier; dropped during cleaning as it carries no predictive information.
Demographics	Age, Gender, Country_Region, Year	Patient age (20–89), gender (3 categories), country/region (10 categories), and the year of diagnosis (2015–2024).
Lifestyle / Environmental	Genetic_Risk, Air_Pollution, Alcohol_Use, Smoking, Obesity_Level	Five numeric risk-factor scores, each ranging roughly 0–10 with a mean close to 5.
Clinical	Cancer_Type, Cancer_Stage, Survival_Years, Target_Severity_Score	Cancer_Type (8 categories: Lung, Prostate, Leukemia, Breast, Liver, Cervical, Colon, Skin); Cancer_Stage is the target variable (Stage 0 / I / II / III / IV); Survival_Years and Target_Severity_Score are continuous clinical indicators.
Treatment	Treatment_Cost_USD	Total treatment cost in US dollars.

Importantly, this dataset is not pre-cleaned: it required duplicate checks, missing-value handling, outlier checks, and encoding of both the target variable and categorical features before it could be used for modelling — satisfying the requirement that the chosen dataset “simulates a real-time problem” and is not delivered in an analysis-ready state.

4. Data Cleaning and Preprocessing

A structured, six-step cleaning pipeline was applied to the raw dataset. Every step was logged so that the effect of each transformation could be verified.

Step 1 — Drop Identifier Columns

The Patient_ID column was removed, as it is a unique identifier with no predictive value. Column names were standardised (whitespace stripped, spaces converted to underscores) for consistency.

Step 2 — Remove Duplicate Rows

The dataset was checked for exact duplicate rows. None were found: the row count remained at 50,000 before and after this step.

Step 3 — Handle Missing Values

Numeric columns were audited for missing values and, where present, filled using the column median (chosen for its robustness to outliers compared to the mean). Categorical columns were filled using the column mode. After this step, a full audit confirmed zero missing cells remained across all 15 columns.

Step 4 — Outlier Removal (IQR Method)

A 1.5× interquartile range (IQR) rule was applied to Age, Survival_Years, and Treatment_Cost_USD — the three continuous features most likely to contain extreme values. The lower and upper bounds were computed and logged for each column. In this dataset, no rows fell outside these bounds, so the dataset size remained at 50,000 rows after this step. This is consistent with the underlying value ranges, which are bounded and fairly uniformly distributed (e.g. Age ranges from 20 to 89 with a roughly flat histogram).

Step 5 — Encode the Target Variable

The target column, Cancer_Stage, was converted from text labels to integers using a LabelEncoder, producing the mapping:

Stage 0	Stage I	Stage II	Stage III	Stage IV
0	1	2	3	4

Step 6 — Encode Categorical Features

The remaining categorical columns — Gender, Country_Region, and Cancer_Type — were each transformed using their own LabelEncoder. Each encoder was saved as a separate .pkl file so the exact same encoding can be re-applied to new data during deployment.

Post-Cleaning Audit

A final data-quality audit confirmed the cleaned dataset retained its full 50,000 rows × 15 columns shape, with zero duplicate rows and zero missing values across every column — ready for exploratory analysis.

5. Exploratory Data Analysis

The exploratory phase investigated the dataset from multiple angles, combining univariate, bivariate, and multivariate visualisations together with statistical summaries. In total, eight distinct plot types were produced, exceeding the minimum of five required by the project brief, and at least six variables were examined (Age, Genetic_Risk, Air_Pollution, Alcohol_Use, Smoking, Obesity_Level, Cancer_Type, Treatment_Cost_USD, Survival_Years, Target_Severity_Score, and Cancer_Stage).

5.1 Target Variable Distribution (Count Plot & Pie Chart)

The first step was to check the balance of the target classes. A count plot and pie chart of Cancer_Stage showed the five stages are almost perfectly balanced — each representing approximately 20% of the dataset (Stage 0: 9,889; Stage I: 10,046; Stage II: 10,124; Stage III: 10,008; Stage IV: 9,933). This balance is favourable for training, since it removes the need for resampling techniques to correct severe class imbalance, although `class_weight='balanced'` was still used as a precaution in the linear and ensemble models.

5.2 Univariate Analysis: Age Distribution (Histogram + KDE)

A histogram of Age showed a roughly uniform distribution between 20 and 89 years, with a mean of 54.4 years. A kernel density estimate (KDE) of Age, split by Cancer_Stage, produced five near-identical, overlapping curves — an early indication that age alone does not strongly differentiate between stages.

5.3 Bivariate Analysis: Box Plots and Violin Plot

Box plots of Age, Survival_Years, and Treatment_Cost_USD grouped by Cancer_Stage were generated to examine how these continuous variables vary across stages. In all three cases, the medians, interquartile ranges, and whisker extents were nearly identical across all five stages. A violin plot of Age by stage confirmed this finding, showing matching distribution shapes for every stage. Group-wise statistical summaries (mean, median, standard deviation) reinforced the same conclusion numerically: stage-conditional means for Age, Survival_Years, and Treatment_Cost_USD differed only marginally from the overall sample means.

5.4 Bivariate Analysis: Cancer Type vs Stage and Treatment Cost by Type

A grouped count plot compared the distribution of Cancer_Stage across the six most common cancer types, and a bar chart showed the mean Treatment_Cost_USD for each of the eight cancer types. Both showed broadly similar patterns across categories — mean treatment cost hovered around \$50,000 regardless of cancer type, and stage distributions within each cancer type tracked the overall ~20%-per-stage pattern.

5.5 Multivariate Analysis: Correlation Heatmap and Pair Plot

A Pearson correlation heatmap was computed across all numeric columns, including the encoded target (Target_Encoded). Two findings stood out:

- Target_Encoded (Cancer_Stage) showed near-zero correlation (approximately -0.01 to $+0.01$) with every other numeric feature in the dataset, including Age, Genetic_Risk, Air_Pollution, Alcohol_Use, Smoking, Obesity_Level, Treatment_Cost_USD, and Survival_Years.
- By contrast, Target_Severity_Score — a separate continuous column also present in the dataset — showed moderate correlations with several lifestyle/environmental features (roughly 0.25 – 0.48 with Genetic_Risk, Air_Pollution, Alcohol_Use, Smoking, and Obesity_Level) and a negative correlation with Treatment_Cost_USD (approximately -0.47).

A pair plot of Age, Survival_Years, and Treatment_Cost_USD, coloured by Cancer_Stage, visually confirmed the heatmap finding: the scatter clouds for all five stages overlap almost completely, with no visible separation between classes.

5.6 EDA Summary

Taken together, the exploratory analysis suggested two important things. First, the dataset is clean, balanced, and well-behaved from a data-quality perspective — there were no missing values, duplicates, or outliers to remove. Second, and more importantly for modelling, none of the available demographic, lifestyle, environmental, or clinical features showed a meaningful univariate or bivariate relationship with Cancer_Stage. This observation foreshadowed the modelling results discussed in Section 8.

6. Feature Engineering and Selection

6.1 New Features

Three new features were engineered from the existing columns, with the aim of capturing relationships that the raw features might not express individually:

Feature	Description
Treatment_Efficiency	$\text{Survival_Years} \div (\text{Treatment_Cost_USD} + 1)$. Intended to capture how much survival benefit a patient gains per dollar spent on treatment (the +1 avoids division by zero).
Is_Senior	Binary flag set to 1 when Age ≥ 60 , else 0. Intended to flag elderly patients, who are hypothesised to present at later stages more often.
Age_Group	Ordinal grouping of Age into four clinically meaningful bands: Young (0–30), Middle (30–45), Senior (45–60), and Elderly (60+).

6.2 Feature Selection

Two complementary feature-selection techniques were applied, satisfying the requirement to use at least one selection method:

1. Filter method — SelectKBest with the ANOVA F-statistic (f_{classif}) was used to score every numeric feature against the encoded target and select the top 10 by F-score.
2. Embedded method — a preliminary Random Forest classifier (100 trees) was trained on the full feature set, and the top 10 features by Gini-based feature importance were extracted.

The final feature set used for modelling was the union of the features selected by both methods, combining the statistical (filter) and model-based (embedded) perspectives to produce a more robust feature list. The encoded categorical variables (Gender, Country_Region, Cancer_Type, Age_Group), the engineered features (Treatment_Efficiency, Is_Senior), and the remaining numeric clinical/lifestyle columns were all candidates for this selection process.

7. Model Building and Tuning

7.1 Train/Test Split

The final feature matrix was split into training and test sets using an 80/20 stratified split (`random_state = 42`), preserving the ~20%-per-class balance in both partitions:

- Training set: 40,000 rows
- Test set: 10,000 rows, with each stage represented by approximately 1,980–2,025 patients

Features were additionally standardised with a `StandardScaler` for the Logistic Regression model, while the tree-based models (Random Forest, XGBoost) were trained on the unscaled feature values, as scaling is not required for tree-based splits.

7.2 Algorithms Used

Three different algorithms were trained and compared, satisfying the requirement that at least three algorithms be attempted:

Algorithm	Configuration
Logistic Regression	Baseline linear classifier. <code>max_iter=1000</code> , <code>class_weight='balanced'</code> , trained on <code>StandardScaler</code> -transformed features.
Random Forest	Ensemble of decision trees. <code>n_estimators=100</code> , <code>class_weight='balanced'</code> . Chosen for its ability to capture non-linear relationships and feature interactions, and for its built-in feature importance scores.
XGBoost	Gradient boosting framework. <code>n_estimators=100</code> , <code>eval_metric='mlogloss'</code> for multi-class classification.

7.3 Initial Model Comparison

Each model was trained on the training set and evaluated on the held-out test set using accuracy, precision, recall, and F1 score (all computed with weighted averaging across the five classes). The results were as follows:

Model	Accuracy	Precision	Recall
Logistic Regression	0.198	0.199	0.198
Random Forest	0.207	0.207	0.207
XGBoost	0.198	0.198	0.198

Note: F1 score (weighted) tracked accuracy closely for all three models (0.196–0.207) and is omitted from this table for brevity; full per-class figures are reported in Section 8.3.

Random Forest produced the highest F1 score of the three baseline models (0.207) and was therefore selected for hyperparameter tuning.

7.4 Hyperparameter Tuning

Hyperparameter tuning is the process of systematically searching over different configurations of a model's parameters (e.g. tree depth, number of estimators) to find the combination that performs best

on validation data. It is important because a model's default settings are rarely optimal for a specific dataset: tuning can reduce overfitting (by limiting model complexity) or underfitting (by allowing more complexity), and generally improves how well a model generalises to unseen data.

Random Forest was tuned using GridSearchCV with 5-fold cross-validation, optimising for the weighted F1 score. The search grid was:

- `n_estimators`: [50, 100, 200]
- `max_depth`: [3, 5, 10, None]
- `min_samples_split`: [2, 5, 10]
- `class_weight`: ['balanced']

This produced 36 candidate combinations, each evaluated across 5 folds (180 model fits in total). The best configuration found was `n_estimators = 50`, `max_depth = None`, `min_samples_split = 2`, `class_weight = 'balanced'`, achieving a mean cross-validated F1 score of 0.2015. This tuned Random Forest was carried forward as the final model.

8. Validation and Evaluation

8.1 What is Validation, and Why Does it Matter?

Validation is the process of estimating how well a trained model will perform on data it has never seen, as opposed to the data it was trained on. A model can achieve very high accuracy on its training data simply by memorising it (overfitting), while performing poorly on new patients in practice. Cross-validation guards against this: the dataset is split into k folds, the model is trained on $k-1$ folds and evaluated on the remaining fold, and this is repeated k times so every fold is used as a test set exactly once. The mean of the k scores gives a more reliable estimate of real-world performance, while the standard deviation indicates how stable that performance is across different subsets of data.

8.2 Cross-Validation Results

The tuned Random Forest model was evaluated with 5-fold stratified cross-validation across the full feature set (X_{final} , y), using four scoring metrics:

Metric	Mean (5-fold CV)	Std. Deviation
Accuracy	0.2006	± 0.0070
Precision (weighted)	0.2005	± 0.0073
Recall (weighted)	0.2006	± 0.0070
F1 (weighted)	0.2001	± 0.0072

The very small standard deviations (≈ 0.007) across folds indicate that the model's performance is highly stable — the result is not an artefact of one unlucky train/test split, but is consistently reproduced across five independent partitions of the data.

8.3 Final Test-Set Results (Tuned Random Forest)

On the held-out 10,000-row test set, the tuned Random Forest achieved an overall accuracy of 0.2022, precision of 0.2020, recall of 0.2022, and F1 score of 0.2017 (all weighted). The per-class breakdown was:

Stage	Precision	Recall	F1 Score	Support
Stage 0	0.19	0.22	0.20	1,978
Stage I	0.21	0.22	0.21	2,009
Stage II	0.22	0.22	0.22	2,025
Stage III	0.21	0.19	0.20	2,002
Stage IV	0.19	0.16	0.18	1,986

8.4 Confusion Matrix

A confusion matrix was generated for the tuned Random Forest on the test set (5×5, one row/column per stage). The matrix showed predictions distributed broadly evenly across all predicted classes for every true class, with the diagonal (correct predictions) only marginally higher than the off-diagonal cells — roughly 19–22% of each true class was predicted correctly, while the remaining 78–81% was spread nearly uniformly across the other four stages. This pattern is the visual signature of a classifier operating close to random chance on a 5-class, balanced problem (where random guessing would itself produce ~20% accuracy and a near-uniform confusion matrix).

9. Discussion: Interpreting the Results

The headline finding of this project is that, despite a complete and methodologically sound pipeline — thorough cleaning, broad exploratory analysis, three engineered features, two feature-selection techniques, three algorithms, and hyperparameter tuning with cross-validation — the final model performs close to the random-guessing baseline for a balanced 5-class problem (~20%). This section examines why.

9.1 Evidence That the Target Carries No Learnable Signal

Three independent pieces of evidence converge on the same conclusion: in this dataset, Cancer_Stage is effectively unrelated to the other 14 columns.

1. Correlation analysis: every numeric feature — including Genetic_Risk, Air_Pollution, Alcohol_Use, Smoking, Obesity_Level, Age, Treatment_Cost_USD, and Survival_Years — has a Pearson correlation with Target_Encoded of approximately -0.01 to +0.01. For comparison, the dataset's other continuous outcome column, Target_Severity_Score, correlates with several of the same features at magnitudes between 0.25 and 0.48, showing that meaningful linear relationships do exist elsewhere in the dataset — just not for Cancer_Stage.

2. Univariate and bivariate plots: histograms, KDE plots, box plots, and violin plots of Age, Survival_Years, and Treatment_Cost_USD, when split by Cancer_Stage, are visually indistinguishable across all five stages. The pair plot shows fully overlapping scatter clouds with no class separation.
3. Model behaviour: a non-linear, interaction-capturing ensemble model (Random Forest) performed only marginally better than a linear baseline (Logistic Regression) — 0.207 vs. 0.198 accuracy — and hyperparameter tuning across 36 configurations produced essentially no improvement (0.2015 best CV F1). If meaningful non-linear or interaction-based signal existed, Random Forest and XGBoost would be expected to substantially outperform Logistic Regression and chance level; instead, all three models converge to $\approx 20\%$, which is exactly the expected accuracy of random guessing on five balanced classes.

9.2 Likely Explanation

The most plausible explanation is that, in this particular Kaggle dataset, the Cancer_Stage column was generated independently of the other patient attributes (e.g. assigned at random or via a process unrelated to the recorded demographic, lifestyle, and clinical variables), whereas Target_Severity_Score appears to have been generated with genuine dependencies on the lifestyle/environmental risk scores. This is a common characteristic of synthetic datasets used for educational purposes, where not every column is guaranteed to be causally linked to every other column.

9.3 Why This Is Still a Valid and Useful Finding

It is important to distinguish between a coding error and a genuine negative result. The pipeline in this project is implemented correctly: data was cleaned appropriately, multiple models were trained and validated correctly, and cross-validation confirms the result is stable rather than due to chance variation in a single split. The exploratory analysis in Section 5 in fact predicted this outcome before any model was trained — the near-zero correlations and overlapping distributions were visible in the EDA stage. A core skill in applied data science is recognising when a dataset does not support the question being asked, rather than over-fitting a model until a misleadingly high score is produced. This project's EDA-to-modelling consistency is, in that sense, a demonstration of sound methodology.

10. Limitations and Recommendations

10.1 Limitations

- Because Cancer_Stage shows near-zero correlation with every available feature, no amount of additional feature engineering, algorithm choice, or hyperparameter tuning is likely to substantially close this gap while using this target column and this dataset.
- The deployed Streamlit application can still accept inputs and return a predicted stage, but users (and graders) should be aware that, on this dataset, the prediction is statistically close to a random draw from the five stage labels.

10.2 Recommendations for Future Work

1. Reframe the prediction target. `Target_Severity_Score`, which correlates meaningfully (0.25–0.48) with several existing features, could be used as a regression target, or discretised into severity bands and used as a classification target instead of `Cancer_Stage`.
2. Source a dataset with a genuinely dependent target. If `Cancer_Stage` prediction remains the goal, a dataset where stage was recorded alongside (and is causally related to) the clinical and lifestyle variables — for example, a real-world clinical registry — would be required.
3. Engineer features from the variables most correlated with `Target_Severity_Score`. Should the target be changed, `Genetic_Risk`, `Air_Pollution`, `Alcohol_Use`, `Smoking`, `Obesity_Level`, and `Treatment_Cost_USD` would be the most promising starting points, based on the correlation analysis in Section 5.5.

11. Deployment

The final tuned Random Forest model, together with its preprocessing artefacts, was deployed as an interactive web application using Streamlit.

Live application: cancer-project-machine-learning.streamlit.app

The app presents a form where a user can enter patient information — age, gender, country/region, cancer type, treatment cost, survival years so far, and diagnosis year — and receive a predicted cancer stage in real time. The following artefacts, produced at the end of the notebook, are loaded by the app:

- `model.pkl` — the tuned Random Forest classifier
- `scaler.pkl` — the `StandardScaler` fitted during training (used for the Logistic Regression path)
- `label_encoder.pkl` — maps the model's integer output back to human-readable stage labels (Stage 0–IV)
- `feature_names.pkl` — the ordered list of features the model expects, ensuring new inputs are encoded and ordered consistently with training

This satisfies the project's deployment requirement of providing a working link to a live web application built on the trained model and its saved transformers.

12. Conclusion

This project implemented a complete, end-to-end machine learning pipeline for the Global Cancer Patients 2015–2024 dataset: a structured six-step data cleaning process, an eight-plot exploratory analysis spanning univariate, bivariate, and multivariate perspectives, three engineered features validated with two feature-selection techniques, three trained algorithms (Logistic Regression, Random Forest, and XGBoost), hyperparameter tuning via `GridSearchCV`, 5-fold cross-validation, and a live Streamlit deployment.

The central research question — whether cancer stage can be predicted from demographic, environmental, lifestyle, and clinical data — was answered empirically and honestly: for this dataset, the answer is no. Cross-validated performance for all three algorithms converges to approximately 20% accuracy, statistically equivalent to random guessing on a balanced 5-class problem, and the project's own internal check confirms this falls short of the required 0.30 precision/recall threshold. Crucially, this result is not attributable to a flaw in the pipeline; it is consistent with, and predicted by, the correlation

analysis and visual exploration carried out in Section 5, and is corroborated by the fact that a non-linear ensemble model could not outperform a linear baseline.

The most constructive path forward, outlined in Section 10, is to either re-target the analysis toward `Target_Severity_Score` — a column that does show meaningful relationships with the available features — or to source a dataset in which `Cancer_Stage` is genuinely dependent on recorded patient attributes. The pipeline, code structure, and deployment built in this project would transfer directly to either of these next steps.

References

- Mughal, Z. (2024). Global Cancer Patients 2015–2024 [Data set]. Kaggle. <https://www.kaggle.com/datasets/zahidmughal2343/global-cancer-patients-2015-2024>
- Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Chen, T. & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.
- Streamlit Inc. (2024). Streamlit Documentation. <https://docs.streamlit.io>